

LuandaPHP 2.0 Documentation

A PHP 8.2 library for programmatic, type-safe HTML generation. No templates, no string concatenation — just clean, structured PHP objects that produce well-indented HTML output.

Table of Contents

- [Installation](#)
 - [Architecture](#)
 - [Page Setup](#)
 - [How Elements Work](#)
 - [Global Attributes](#)
 - [Global Events](#)
 - [Structural Elements](#)
 - [Text & Inline Formatting](#)
 - [Form Elements](#)
 - [Table Elements](#)
 - [Media & Embedded Elements](#)
 - [Interactive Elements](#)
 - [Lists](#)
 - [Scripting](#)
 - [Incident Reporter](#)
 - [Constants Reference](#)
 - [Full Example](#)
-

Installation

```
composer require tamasvarga/luandaphp
```

Requires PHP 8.2 or higher. No additional dependencies.

Architecture

LuandaPHP is built on a strict inheritance chain. Every HTML element ultimately extends from `Element`, gaining access to escaping helpers. Most elements extend `Node`, which gives them content management, indentation, and the full global attribute and event system.

Element

- └─ GlobalEvent (all HTML event attributes: onclick, onchange, etc.)
 - └─ GlobalAttr (all HTML global attributes: id, class, style, etc.)
 - └─ Node (content management, indentation, Show())
 - └─ All HTML element classes (Div, Span, Paragraph, etc.)

`Element` is the base abstract class. It provides:

- `safeHtml(string $text): string` — escapes output for safe HTML content and attributes
- `safeUrl(string $url): string` — escapes URLs for use in href, src, and action attributes
- `hasValue(mixed $value): bool` — checks whether a value is non-null and non-empty, used throughout for conditional attribute output

`GlobalEvent` declares every standard HTML event attribute as a protected nullable string property. `GlobalAttr` adds every HTML global attribute on top of that. `Node` adds content management via `HtmlContent` and the `setLevel()` / `getHtml()` / `Show()` contract defined by `INodeInterface`.

INodeInterface

Every element that can be placed inside another must implement `INodeInterface`:

```
interface INodeInterface {  
    public function setLevel(int $level): void;  
    public function getHtml(): string;  
}
```

This is what allows any element to be nested inside any other via `addContent()`.

Page Setup

The `Html` class is the entry point for every page. It manages the `<html>`, `<head>`, and `<body>` elements internally.

```
$page = new Html('My Page Title');
```

Common setup methods

```
$page->setLang('en');           // Sets the lang attribute on <html>, default 'en'
```

```

$page->setCharset();           // Adds <meta charset="UTF-8"> — accepts charset::
constants
$page->setupMobile();          // Adds viewport meta tag for responsive design
$page->setNoCache();           // Adds cache-control meta tags

$page->setupTailwind();        // Loads Tailwind CSS via CDN
$page->setupBootstrap();       // Loads Bootstrap 5 CSS and JS via CDN
$page->setupFontAwesome();     // Loads Font Awesome 6 free tier via CDN

```

Head management

```

$page->addStylesheet('https://example.com/style.css'); // Adds a <link rel="stylesheet">
$page->addStylesheet('print.css', 'print');           // With optional media query
$page->addLink(new Resource('preload', 'font.woff2')); // Generic resource link
$page->addMeta(new Meta('description', 'My page description')); // Arbitrary meta tag
$page->changeTitle('New Title');                     // Updates the page title
$page->setBaseUrl('https://example.com/');           // Sets <base href>
$page->setBaseTarget('_blank');                      // Sets <base target>

```

Script management

Scripts are placed using `script_type` constants:

```

$page->addScript(script_type::HEADLINK, 'https://example.com/lib.js'); // External script in
<head>
$page->addScript(script_type::BODYLINK, 'https://example.com/app.js'); // External script at
end of <body>
$page->addScript(script_type::RUNCMD, 'console.log("ready")');          // Inline script at end
of <body>

```

Adding content and rendering

```

$page->addContent($element); // Adds any INodeInterface element to <body>
$page->Show();                // Outputs the complete HTML document to the browser

```

How Elements Work

Every element class follows the same pattern:

```

$div = new Div();
$div->setId('main');
$div->addClass('container mx-auto');
$div->addContent(new Paragraph());

```

```
echo $div->getHtml();
```

Content nesting

Elements are nested using `addContent()`. Any element that implements `INodeInterface` can be passed:

```
$section = new Section();  
$heading = new Heading(2);  
$heading->addContent(new Text('Hello World'));  
$section->addContent($heading);
```

Cloning

If you need the same element structure repeated with independent state, use `addClone()`:

```
$template = new ListItem('Default');  
$list->addClone($template); // Adds a deep clone, original is unaffected
```

Indentation

Indentation is managed automatically. The `Html` class starts `<head>` and `<body>` at level 1, and each nested element increments the level. You never set indentation manually — it propagates through `setLevel()` calls during `getHtml()`.

Direct output

Any `Node`-based element can be output directly without going through `Html`:

```
$div->Show(); // Equivalent to echo $div->getHtml()
```

Global Attributes

All element classes inherit the full set of HTML global attributes from `GlobalAttr`. These are available on every element:

```
$el->setId('my-id');  
$el->addClass('class-one class-two'); // Space-separated, stored as unique set  
$el->setStyle('color: red;');  
$el->setTitle('Tooltip text');  
$el->setName('field-name');  
$el->setTabIndex(1);
```

```

$el->setLanguage('hu');
$el->setDirection(text_direction::LEFT); // ltr, rtl, auto
$el->setAccesskey('k');
$el->setPopover(popover::AUTO);          // auto or manual
$el->toTranslate(translate::YES);        // yes or no
$el->toSpellcheck();
$el->toEditable();                       // contenteditable
$el->setDraggable();
$el->setAutoFocus();
$el->setInert();
$el->Hide();                             // hidden attribute
$el->Disable();                           // disabled attribute
$el->addAttr('data-custom', 'value');    // Any arbitrary attribute

```

`getClasses()` and `getAttributes()` are called internally by every element's `getHtml()` method — you never call them yourself.

Global Events

All elements also inherit every standard HTML event attribute from `GlobalEvent`. Events are added using `addEvent()` with event name constants:

```

$button->addEvent(mouse_events::CLICK, 'handleClick());
$input->addEvent(input_events::CHANGE, 'handleChange(this)');
$form->addEvent(input_events::SUBMIT, 'return validate()');
$div->addEvent(drag_events::DROP, 'handleDrop(event)');
$window->addEvent(window_events::RESIZE, 'onResize()');

```

If you attempt to add a duplicate event to the same element, `IncidentReporter` logs it and the original is preserved. If you attempt an invalid event name, it is also logged.

Event constant classes

Class	Covers
<code>mouse_events</code>	click, dblclick, mousedown, mouseup, mousemove, mouseenter, mouseleave, mouseover, mouseout, contextmenu, auxclick, wheel, scroll, scrollend
<code>keyboard_events</code>	keydown, keyup, keypress (deprecated)

<code>input_events</code>	input, beforeinput, change, invalid, submit, reset, select, search, formdata
<code>focus_events</code>	focus, blur, focusin, focusout
<code>pointer_events</code>	pointerdown, pointerup, pointermove, pointerenter, pointerleave, pointerover, pointerout, pointercancel, gotpointercapture, lostpointercapture
<code>touch_events</code>	touchstart, touchend, touchmove, touchcancel
<code>drag_events</code>	dragstart, drag, dragenter, dragover, dragleave, drop, dragend
<code>clipboard_events</code>	copy, cut, paste
<code>animation_events</code>	animationstart, animationend, animationiteration, animationcancel
<code>media_events</code>	play, pause, ended, volumechange, timeupdate, loadeddata, and more
<code>window_events</code>	load, unload, resize, hashchange, popstate, online, offline, storage, and more
<code>misc_events</code>	toggle, close, cancel, command, readystatechange, visibilitychange, and more

Structural Elements

These are block-level container elements. All accept `addContent()` and support the full global attribute and event system.

```

new Div()      // Generic block container
new Span()     // Generic inline container
new Section()  // Thematic section
new Article()  // Self-contained content
new Aside()    // Sidebar / tangentially related content
new Header()   // Introductory content or navigation
new Footer()   // Footer content
new Main()     // Main content area — use once per page
new Nav()      // Navigation links
new Paragraph() // <p> block of text
new Blockquote() // Block quotation, supports cite attribute
new Address()   // Contact information
new Hr()       // Thematic break — self-closing
new Br()       // Line break — self-closing

```

Headings

```
$h = new Heading(1); // Accepts 1–6, produces <h1>–<h6>
$h->addContent(new Text('Title'));
```

Text & Inline Formatting

Text

Text is the primary inline content node. By default it outputs raw escaped text with no wrapping tags:

```
new Text('Hello world')           // Outputs: Hello world
new Text('Hello', text_format::STRONG) // Outputs: <strong>Hello</strong>
```

Formatting is controlled via a bitmask using **text_format** constants. Multiple formats can be combined with the **|** operator:

```
new Text('Important', text_format::STRONG | text_format::EM | text_format::MARK)
// Outputs: <strong><em><mark>Important</mark></em></strong>
```

Formats can also be set or added after construction:

```
$t = new Text('Hello');
$t->setFormat(text_format::STRONG); // Replaces current format
$t->addFormat(text_format::EM);    // Adds to current format
```

text_format constants

Constant	Tag	Meaning
<code>text_format::STRONG</code>	<code></code>	Important text
<code>text_format::EM</code>	<code></code>	Emphasis
<code>text_format::MARK</code>	<code><mark></code>	Highlighted / relevant
<code>text_format::B</code>	<code></code>	Stylistic bold

<code>text_format::I</code>	<code><i></code>	Stylistic italic
<code>text_format::U</code>	<code><u></code>	Underline annotation
<code>text_format::S</code>	<code><s></code>	Strikethrough
<code>text_format::DEL</code>	<code></code>	Deleted text
<code>text_format::INS</code>	<code><ins></code>	Inserted text
<code>text_format::SMALL</code>	<code><small></code>	Side comment / fine print
<code>text_format::SUB</code>	<code><sub></code>	Subscript
<code>text_format::SUP</code>	<code><sup></code>	Superscript
<code>text_format::KBD</code>	<code><kbd></code>	Keyboard input
<code>text_format::SAMP</code>	<code><samp></code>	Sample output
<code>text_format::VAR</code>	<code><var></code>	Variable
<code>text_format::DFN</code>	<code><dfn></code>	Definition term
<code>text_format::Q</code>	<code><q></code>	Inline quotation
<code>text_format::CODE</code>	<code><code></code>	Inline code

Additional Text methods

```
$t->setText('New content');      // Replaces text (escaped)
$t->addText(' appended');        // Appends text (escaped)
$t->getText();                    // Returns raw stored text
$t->getFromUrl('https://...');    // Fetches remote content and sets as text
```

Pre

Pre is a block-level preformatted element. It outputs a leading newline before the opening tag but **no newlines inside** — whitespace inside `<pre>` is rendered literally by the browser, so content is responsible for its own line breaks:

```
$pre = new Pre();
$pre->addContent(new Text("line one\nline two\nline three"));
```

Bdi and Bdo

Bidirectional text control elements:

```
$bdi = new Bdi(); // Isolates text direction automatically
$bdi->setDir(text_direction::AUTO);

$bdo = new Bdo(text_direction::RIGHT); // Overrides text direction to rtl
$bdo->addContent(new Text('Hello')); // Renders as: olleH
```

Form Elements

Form

```
$form = new Form();
$form->setMethod(form_method::POST); // get or post
$form->setAction('/submit.php');
$form->setEnctype(form_enctype::FILE); // For file uploads
$form->setAutocomplete(form_autocomplete::OFF);
$form->setNovalidate();
$form->setTarget('_blank');
```

Input

```
$input = new Input(form_input_type::TEXT);
$input->setName('username');
$input->setPlaceholder('Enter username');
$input->setRequired();
$input->setReadonly();
$input->setMinMaxLen(3, 50);
$input->setMinMax(1, 100); // For number/range inputs
$input->setStep(0.5);
$input->setPattern('[A-Za-z]+');
$input->setAccept(form_file_format::IMAGE); // For file inputs
$input->setMultiple(); // For file/email inputs
$input->value = 'default'; // Public — set directly
```

form_input_type constants

BTN, CHKBOX, COLORPICKER, DATE, LCLDATE, EMAIL, FILE, HIDDEN, IMG, MONTH, NUM, PWD, RADIOBTN, RANGE, RESET, SEARCH, SUBMIT, TEL, TEXT, TIME, URL, WEEK

Label

```
$label = new Label('Email Address');  
$label->setFor('email-input-id'); // Links label to input via for attribute
```

Button

```
$btn = new Button('Click me');  
$btn->setType(form_button_type::SUBMIT); // button, reset, submit  
$btn->setForm('form-id'); // Associates with external form
```

Textarea

```
$ta = new Textarea();  
$ta->setName('message');  
$ta->setSize(40, 8); // cols, rows  
$ta->setPlaceholder('Type here...');  
$ta->setMinMaxLen(10, 500);  
$ta->setRequired();  
$ta->setReadonly();  
$ta->setWrap(textarea_wrapmode::SOFT); // soft or hard  
$ta->setValue('Default content');
```

Select and Option

```
$select = new Select();  
$select->setName('country');  
$select->setSize(5);  
$select->setMultiple();  
$select->setRequired();
```

```
$opt = new Option('gb', 'United Kingdom');  
$opt->Select(); // Marks as selected  
$opt->Deselect(); // Removes selected state  
$opt->isSelected(); // Returns bool
```

```
$select->addOption($opt);  
$select->getSelected(); // Returns index of selected option  
$select->getSelectedAll(); // Returns array of all selected indices
```

Table Elements

```
$table = new Table('Optional Caption Text'); // Caption is optional
$table->setCaption(new Caption('My Table')); // Or set explicitly

 $thead = new THead(); |  | | --- | | $tbody = new TBody(); | | $tfoot = new TFoot(); | | $row = new TRow(); | | $th = new THeader(); | | $th->setScope(th_scope::COL); // row, col, rowgroup, colgroup, auto | | $th->setColspan(2); | | $th->setRowspan(2); | | $th->setAbbr('Abbreviated label'); | | $td = new TCell(); | | $td->setColspan(2); | | $td->setRowspan(2); | | $td->setHeaders('th-id-1 th-id-2'); | | $colgroup = new ColGroup(3); // Optional span count | | $col = new Col(2); // Optional span count | |
```

Media & Embedded Elements

Img

```
$img = new Img('photo.jpg', 'A description'); // src, alt
$img->setSize(800, 600); // width, height
$img->setCrossorigin('anonymous');
$img->setLoading('lazy');
$img->setDecoding('async');
```

Figure and FigCaption

```
$figure = new Figure();
$figure->addContent($img);
$caption = new FigCaption();
$caption->addContent(new Text('Photo caption'));
$figure->addContent($caption);
```

Meter

```
$meter = new Meter(0.75);      // Current value, default 0
$meter->setValue(0.9);
$meter->setThresholds(0.3, 0.8); // low, high
$meter->setOptimum(0.5);
$meter->addAttr('min', '0');
$meter->addAttr('max', '1');
```

Time

```
$time = new Time('2026-04-29');      // Optional datetime in constructor
$time->setDatetime('14:30');
$time->addContent(new Text('29 April 2026')); // Human-readable display

// Content can be anything — including SVG
$time->addContent($svgClockElement);
```

Script

For direct script element creation (usually managed through `Html::addScript()`):

```
$script = new Script('https://example.com/lib.js');
$script->setOrigin('anonymous'); // crossorigin attribute

$inline = new Script('console.log("hello")');
$inline->runScript();           // Returns inline <script> tag output
```

Interactive Elements

Details and Summary

```
$details = new Details();
$summary = new Summary();
$summary->addContent(new Text('Click to expand'));
$details->addContent($summary);
$details->addContent(new Paragraph());
```

Ruby Annotation

Used for East Asian typography — shows phonetic annotations above base characters:

```
$ruby = new Ruby();
$ruby->addContent(new Text('漢'));
```

```
$rp1 = new Rp();
$rp1->addContent(new Text(''));
$ruby->addContent($rp1);

$rt = new Rt();
$rt->addContent(new Text('かん'));
$ruby->addContent($rt);

$rp2 = new Rp();
$rp2->addContent(new Text(''));
$ruby->addContent($rp2);
```

Browsers that support ruby display かん above 漢. Browsers that don't show 漢(かん).

Lists

```
$ul = new HtmlList(list_style::UNORDERED); // <ul>
$ol = new HtmlList(list_style::ORDERED);   // <ol>
$dl = new HtmlList(list_style::DESCRIPTION); // <dl>

$ul->addContent(new ListItem('First item'));
$ul->addContent(new ListItem('Second item'));

// Description list uses term and data types
$dl->addContent(new ListItem('Term', listitem_type::TERM));
$dl->addContent(new ListItem('Definition', listitem_type::DATA));
```

Scripting

Resource (link element)

```
$link = new Resource('stylesheet', 'style.css');
$link = new Resource('preload', 'font.woff2', 'screen'); // rel, url, media
$page->addLink($link);
```

Meta

```
$meta = new Meta('description', 'Page description'); // name, content
$meta = new Meta('og:title', 'Open Graph Title');

$charset = new Meta();
$charset->setCharset(charset::UTF8);
```

```
$page->addMeta($meta);
```

Incident Reporter

IncidentReporter is a static internal logging class used by the library to record errors such as duplicate events or invalid attribute names. You can also use it in your own application code.

```
use TamasVarga\LuandaPHP\Misc\IncidentReporter;
```

```
// Check if available (always true when the class is loaded)
IncidentReporter::isAvailable();
```

```
// Log an entry
```

```
IncidentReporter::report('MyClass', 'Something went wrong', 500);
```

```
// Stored as: [14:32:01] Origin: MyClass | Error: Something went wrong | Code: 500
```

```
// Retrieve all logs
```

```
$logs = IncidentReporter::getLogs(); // Returns array of formatted strings
```

```
// Write logs to file (appends)
```

```
IncidentReporter::writeToFile('/var/log/luanda.log');
```

```
// Get a self-contained HTML report
```

```
$html = IncidentReporter::toHtml('My Report');
```

Constants Reference

Class	Location	Purpose
<code>text_format</code>	Text.php	Bitmask flags for inline text formatting
<code>form_input_type</code>	GlobalAttr.php	Input type values (TEXT, EMAIL, NUM, etc.)
<code>form_method</code>	GlobalAttr.php	Form method values (GET, POST)
<code>form_ctype</code>	GlobalAttr.php	Form encoding types
<code>form_autocomplete</code>	GlobalAttr.php	Autocomplete on/off

<code>form_button_type</code>	GlobalAttr.php	Button types (BTN, RESET, SUBMIT)
<code>form_rel</code>	GlobalAttr.php	Link relation values
<code>form_file_format</code>	GlobalAttr.php	File accept types (IMAGE, PDF, etc.)
<code>list_style</code>	GlobalAttr.php	List types (UNORDERED, ORDERED, DESCRIPTION)
<code>listitem_type</code>	GlobalAttr.php	List item types (ITEM, TERM, DATA)
<code>script_type</code>	GlobalAttr.php	Script placement (HEADLINK, BODYLINK, RUNCMD)
<code>textarea_wrap_mode</code>	GlobalAttr.php	Textarea wrap (SOFT, HARD)
<code>th_scope</code>	Table.php	Table header scope values
<code>text_direction</code>	GlobalAttr.php	Text direction (LEFT, RIGHT, AUTO)
<code>translate</code>	GlobalAttr.php	Translate attribute (YES, NO)
<code>popover</code>	GlobalAttr.php	Popover state (AUTO, MANUAL)
<code>charset</code>	GlobalAttr.php	Character set values (UTF8, UTF16, etc.)
<code>indent_type</code>	GlobalAttr.php	Indentation characters (TAB, SPACE, etc.)
<code>special_chars</code>	GlobalAttr.php	Unicode constants (NEWLINE, NBSP, NDASH, etc.)
<code>mouse_events</code>	GlobalEvent.php	Mouse event attribute names
<code>keyboard_events</code>	GlobalEvent.php	Keyboard event attribute names
<code>input_events</code>	GlobalEvent.php	Form/input event attribute names
<code>focus_events</code>	GlobalEvent.php	Focus event attribute names
<code>pointer_events</code>	GlobalEvent.php	Pointer event attribute names

<code>touch_events</code>	GlobalEvent.php p	Touch event attribute names
<code>drag_events</code>	GlobalEvent.php p	Drag and drop event attribute names
<code>clipboard_events</code>	GlobalEvent.php p	Copy/cut/paste event attribute names
<code>animation_events</code>	GlobalEvent.php p	CSS animation event attribute names
<code>media_events</code>	GlobalEvent.php p	Media playback event attribute names
<code>window_events</code>	GlobalEvent.php p	Window and document event attribute names
<code>misc_events</code>	GlobalEvent.php p	Miscellaneous DOM event attribute names

Full Example

```
<?php
declare(strict_types=1);

require_once 'vendor/autoload.php';

use TamasVarga\LuandaPHP\Html;
use TamasVarga\LuandaPHP\Section;
use TamasVarga\LuandaPHP\Heading;
use TamasVarga\LuandaPHP\Paragraph;
use TamasVarga\LuandaPHP\Div;
use TamasVarga\LuandaPHP\Text;
use TamasVarga\LuandaPHP\Form;
use TamasVarga\LuandaPHP\Input;
use TamasVarga\LuandaPHP\Button;
use TamasVarga\LuandaPHP\text_format;
use TamasVarga\LuandaPHP\form_input_type;
use TamasVarga\LuandaPHP\form_method;
use TamasVarga\LuandaPHP\mouse_events;

// Page setup
$page = new Html('My First Page');
$page->setCharset();
$page->setupMobile();
$page->setupTailwind();
```

```
// Section
$section = new Section();
$section->addClass('max-w-xl mx-auto py-12 px-6');

// Heading
$h1 = new Heading(1);
$h1->addClass('text-3xl font-bold mb-4');
$h1->addContent(new Text('Hello ', text_format::STRONG));
$h1->addContent(new Text('World', text_format::STRONG | text_format::MARK));
$section->addContent($h1);

// Paragraph
$p = new Paragraph();
$p->addClass('text-gray-600 mb-6');
$p->addContent(new Text('Built with LuandaPHP 2.0.'));
$section->addContent($p);

// Form
$form = new Form();
$form->setMethod(form_method::POST);
$form->addClass('flex flex-col gap-4');

$input = new Input(form_input_type::EMAIL);
$input->setName('email');
$input->setPlaceholder('your@email.com');
$input->addClass('border rounded px-3 py-2');
$input->setRequired();
$form->addContent($input);

$btn = new Button('Submit');
$btn->addClass('bg-blue-600 text-white px-4 py-2 rounded');
$btn->addEvent(mouse_events::CLICK, 'alert("Submitted!")');
$form->addContent($btn);

$section->addContent($form);
$page->addContent($section);

$page->Show();
```